Design Planning Doc

We are planning to implement a Sudoku game with console interface and extend it with a GUI in case we have some time left.
In addition to the necessary required classes (Game class, Player class, User Interface class) we want to have a Sudoku class which will represent the players' Sudoku field. We also want to have a persistence class to save and load the game or high score.
Anja is going to implement the Sudoku class and game class and Serafina the user_interface and persistence class. We will work together on the player class. If we encounter any issues we will help each other and find a solution together.

Game class

The game class controls the game flow as well as the Sudoku field. Note that a few required methods were renamed or put into other classes for better clarity and distribution.

__init__(size, difficulty): The init method in the game class will create a new game session and call the constructor in the Sudoku class. The number of players in Sudoku is always one, therefore the only parameters here will be the difficulty (easy, medium and difficult) and the size of the Sudoku field (per default 9 with 3x3 grids). To be able to set a high score we also want to implement a timer that measures the needed time.

start_timer(): This is a helper method to start a timer a soon as a Game is created.

set_player(player): Sets a given Player object as the player for the current game session. An add_player() method that was suggested in the given Game class code is not applicable in a Sudoku Game since there is only one player.

finish_game(): Cleans-up code after a game session ends.

is_finished() → bool : Checks if the game is successfully finished by calling the method is_solved() in Sudoku class. This is the renamed win_condition() method.

lose_condition(): In Sudoku a player can not really lose. That is why it is not appointed yet if we want to implement this method. The only thing it could possibly check is if a Sudoku is not solvable, but that should not happen since the program should be able to automatically generate new, and always solvable Sudoku fields.

step(players_input) → bool : One player input in a Sudoku cell, which is in the default Sudoku a number between 1 and 9. Also the player should have the possibility to erase his own previous inputs. The return value represents if the step was valid and successful.

get_current_state() : Returns the current Sudoku field of the game session.

run(): Runs a game session interactively until the win condition is reached.

Sudoku class

The Sudoku class represents a players' Sudoku field and consists of following functions:

__init__(size: int = 9): Initializes objects of Sudoku class with default size of 9 fields as height and width. Each field consists of grids which are per default 3x3.

create(difficulty): Generates a new Sudoku that is solvable and has some empty entries depending on difficulty (easy, medium or difficult).

set(x, y, new_value): Sets specified new_value in a the Sudoku field with coordinates x and y and allows 'empty' value 0.

is_valid_coordinate(x: int, y: int) -> bool : Checks if given coordinates x and y lie in a Sudoku field.

solve(): Solves complete Sudoku game through a backtracking algorithm.

is_filled(): Checks if the Sudoku field is already completely filled.

is_valid(): Checks if all Sudoku field entries are valid. According to the rules there must not be any duplicate in every row and column, as well as in each grid.

is_solved(): Checks if the Sudoku field is completely solved.

remove(x, y): Removes an entry in a Sudoku field and replaces it by invalid value 0.

__str__(): This method gives the string representation of a Sudoku field as a formatted matrix and converts empty '0' values to gaps.

Furthermore, if we have time left Anja wants to implement a get_a_hint(x,y) method which returns a correct value in a random or given field and show_wrong_fields() that compares the current Sudoku values to a possible solution and returns which are different and probably false (if there is only exact one solution).

Player class
__init__(name, password) -> initializes a new player, optional with a connected password. The method can also create a new player without a password, which you can add later

add_password(password) -> bool: adds a password for the current player

log_in(name,password) -> bool: checks if the name matches the password

update_highscore(new_highscore) -> bool: updates the current high score with a new high score

UserInterface class
This class will be used for any inputs and outputs

__init__(): the init method will start the whole user interface interaction on the console, it will be one of the first methods to call in the main function, and will probably start with the instructions printed on the console

give_instruction(filename): Will print a welcome text and instructions to the console

display(): will print the current game on the console

frame(): will probably not be used

wait_for_input(): not needed, functionality is included in read_input()

read_input(): processes user input, x and y coordinate and the value (probably with a helper method) so we can call the set method from the Sudoku class with the given input. You can also delete an entry or change an already existing entry. It will throw an exception when you try to put a number into the wrong field. It may will be possible to get a hint for solving the Sudoku (if it's too hard for the player). You can also ask for the instructions how to play Sudoku again.

Persistence class
save_highscore(scores, filename) -> Bool: Will save all current high scores into the file
load_highscore(player, filename) -> Float: Returns the current high score from a json file
save_game(game, filename) -> Bool: Will save the game as a json file
load_game(cls, filename) -> Game: Will load the entire game from a previous saved json file